

Actividad 3: Diseño y dominio

Objetivo

Diseñar un núcleo de negocio que proteja reglas e invariantes, defina límites de dominio y conecte casos de uso con infraestructura mediante puertos y adaptadores. La solución debe poder cambiar de base de datos, API externa o framework sin obligar a reescribir las reglas principales.

Entregables obligatorios

- Enlace al repositorio de GitHub con el resultado de la actividad, documentos, código e historial de commits.
- Enlace al video de sustentación publicado en YouTube, Google Drive o Microsoft Teams.
- No adjuntar el archivo de video directamente; entregar únicamente su enlace de acceso.

Alcance de la actividad

La actividad se trabaja sobre la plataforma logística de última milla. El estudiante debe implementar un flujo vertical pequeño, por ejemplo asignar una entrega, que atraviese aplicación, dominio, persistencia e integración sin mezclar responsabilidades.

¿Qué se evalúa?

- modelo de dominio,
- entidades y objetos de valor,
- invariantes y reglas del negocio,
- casos de uso,
- contratos de aplicación,
- puertos y adaptadores,
- separación entre dominio, aplicación e infraestructura,
- dirección correcta de dependencias,
- decisiones de diseño con trade-offs explícitos,
- código organizado y ejecutable dentro de `/src`.

Conceptos clave

1. Dominio

Es la parte del sistema que representa el negocio: reglas, conceptos, procesos, restricciones, políticas.

2. Entidad

Es un objeto con identidad propia.

Ejemplo:

- cliente,
- pedido,
- empleado,

- reserva.

3. Objeto de valor

No tiene identidad propia; se describe por sus atributos.

Ejemplo:

- monto,
- dirección,
- fecha de entrega,
- prioridad.

4. Invariante

Es una regla que siempre debe cumplirse.

Ejemplo:

- una reserva no puede superarse en capacidad,
- un pedido no puede estar en dos estados contradictorios,
- un usuario no puede tener dos roles conflictivos.

Principios importantes

SOLID

- S: responsabilidad única,
- O: abierto/cerrado,
- L: sustitución de Liskov,
- I: segregación de interfaces,
- D: inversión de dependencias.

Inversión de dependencias

Las capas internas no deberían depender de elementos externos. El dominio debería estar protegido.

Puertos y adaptadores

- **Puerto:** interfaz que expresa lo que el caso de uso necesita, por ejemplo `IOrderRepository` o `IRouteEstimator`.
- **Adaptador:** implementación concreta para una base de datos, API, cola o servicio externo.
- **Regla:** el dominio y la aplicación definen la necesidad; la infraestructura implementa el detalle.

Estructura recomendada

- `/src/Domain`: entidades, objetos de valor, invariantes y reglas.
- `/src/Application`: casos de uso, DTOs, contratos y puertos.
- `/src/Infrastructure`: repositorios, APIs externas, notificaciones y adaptadores.
- `/tests`: pruebas unitarias del dominio y pruebas de integración del flujo.

Casos de uso y contratos

Un caso de uso responde a la pregunta: ¿qué necesita hacer el usuario o el sistema?

Ejemplo:

- Crear reserva,
- Cancelar reserva,
- Confirmar pago,
- Obtener historial de pedidos.

Cada caso de uso debe tener una intención clara, sin mezclarse con detalles de base de datos o HTTP.

El caso de uso debe recibir sus dependencias por constructor y depender de interfaces. No debe crear directamente `SqlConnection`, `HttpClient` configurado para un proveedor concreto ni clases de infraestructura.

Ejemplo de modelo de dominio

Para la plataforma logística de última milla:

- Cliente
- Pedido
- LíneaPedido
- Inventario
- Almacén
- Ruta
- Repartidor
- EstadoEntrega
- Incidencia
- Devolución

Reglas:

- un pedido no puede confirmarse si el stock solicitado no está disponible,
- una entrega no puede asignarse a un repartidor si la ruta ya está saturada,
- una incidencia no puede cerrarse sin registrar resolución ni responsable,
- una devolución debe estar asociada a un pedido válido y a un motivo concreto.

Caso de uso obligatorio sugerido

Implementar `AssignCourierUseCase`:

1. Cargar la entrega mediante `IDeliveryRepository`.
2. Verificar que la entrega exista y no tenga repartidor activo.
3. Consultar disponibilidad mediante un puerto, no mediante una API concreta.
4. Ejecutar `delivery.AssignCourier(courierId)` para proteger la invariante.
5. Guardar mediante el repositorio.
6. Publicar o registrar el resultado mediante un adaptador.

Ejemplo mínimo en C#

```
public sealed record DeliveryId(Guid Value);
public sealed record DeliveryAddress(string Value);

public sealed class Delivery
{
    public DeliveryId Id { get; }
    public DeliveryAddress Address { get; }
    public Guid? CourierId { get; private set; }

    public void AssignCourier(Guid courierId)
    {
        if (CourierId.HasValue)
            throw new InvalidOperationException("Delivery already assigned");

        CourierId = courierId;
    }
}

public interface IDeliveryRepository
{
    Task<Delivery?> GetAsync(DeliveryId id);
    Task SaveAsync(Delivery delivery);
}

public sealed class AssignCourierUseCase
{
    private readonly IDeliveryRepository _repository;

    public AssignCourierUseCase(IDeliveryRepository repository)
    {
        _repository = repository;
    }

    public async Task ExecuteAsync(DeliveryId deliveryId, Guid courierId)
    {
        var delivery = await _repository.GetAsync(deliveryId)
            ?? throw new InvalidOperationException("Delivery not found");

        delivery.AssignCourier(courierId);
        await _repository.SaveAsync(delivery);
    }
}
```

En este ejemplo, la entidad protege la invariante, el caso de uso coordina la intención y el repositorio es un puerto. La implementación de base de datos queda en infraestructura.

Diagramas que puedes incluir

- diagrama de componentes,
- diagrama de capas,
- diagrama de dependencias,
- mapa de entidades y relaciones.

Video de sustentación

Explica:

- qué es el dominio,
- cuántas capas usaste y por qué,
- dónde están las reglas críticas,
- cómo evitas acoplar el negocio a la infraestructura,
- qué decisiones de diseño fueron clave,

- qué puertos y adaptadores implementaste,
- cómo una prueba demuestra una invariante.

Checklist final

- ☐ modelo de dominio definido
- ☐ entidades y objetos de valor diferenciados
- ☐ invariantes listados
- ☐ casos de uso principales descritos
- ☐ contratos de aplicación definidos
- ☐ puertos y adaptadores implementados
- ☐ dependencias dirigidas correctamente
- ☐ código organizado en `/src`
- ☐ pruebas unitarias de reglas críticas
- ☐ diagrama de capas o componentes
- ☐ README con instrucciones de ejecución
- ☐ video entregado

Consejo importante

La lógica de negocio debe poder cambiar sin depender de una base de datos, una API externa o un framework concreto.